

DS Lab-02

Muhammad Aun Anis, 24P-3017

Task 1:

```
#include<iostream>
using namespace std;
void rowSum(int **, int, int);
void colSum(int **, int, int);

int main(){
    cout<<"24P-3017"<<endl;

    int r, c;
    cout<<"\nEnter the number of rows: "; cin>>r;
    cout<<"Enter the number of columns: "; cin>>c; cout<<endl;

    int **array=new int*[r];
    for(int i=0;i<r;i++){
        *(array+i)=new int[c];
    }

    for (int i=0;i<r;i++) {
        for (int j=0;j<c;j++) {
            *((array+i)+j)=rand()%100;
        }
    }

    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++){
            cout<<*((array+i)+j)<<" ";
        }
        cout<<endl;
    }

    cout<<"\nRow Sum:\n"; rowSum(array, r, c);
    cout<<"\nColumn Sum:\n"; colSum(array, r, c);

    for(int i=0;i<r;i++){
        delete[] *(array+i);
```

```

    }
    delete[] array;
}

void rowSum(int **array, int rowSize, int columnSize){
    int rowSum=0;
    for(int i=0;i<rowSize;i++){
        cout<<"row #"<<i+1<<": ";
        for(int j=0;j<columnSize;j++){
            rowSum+=*(array+i+j);
        }
        cout<<rowSum<<endl;
        rowSum=0;
    }
}

void colSum(int **array, int rowSize, int columnSize){
    int colSum=0;
    for(int i=0;i<rowSize;i++){
        cout<<"column #"<<i+1<<": ";
        for(int j=0;j<columnSize;j++){
            colSum+=*(array+j+i);
        }
        cout<<colSum<<endl;
        colSum=0;
    }
}

```

Output:

```

24P-3017

Enter the number of rows: 3
Enter the number of columns: 3

41 67 34
0 69 24
78 58 62

Row Sum:
row #1: 142
row #2: 93
row #3: 198

Column Sum:
column #1: 119
column #2: 194
column #3: 120

```

Task 2:

```
#include<iostream>
using namespace std;
int *findIntersection(int *, int *, int, int, int*);

int main(){
    cout<<"24P-3017"<<endl;

    int size1=5;
    int size2=7;
    int array1[size1]={1, 3, 6, 8, 56};
    int array2[size2]={1, 1, 6, 7, 8, 8, 45};

    int resultSize;
    int *resultArray=findIntersection(array1, array2, size1, size2, &resultSize);

    cout<<"\nA = ";
    for(int i=0;i<size1;i++){
        cout<<array1[i]<<" ";
    }
    cout<<endl;
    cout<<"B = ";
    for(int i=0;i<size2;i++){
        cout<<array2[i]<<" ";
    }
    cout<<endl;

    cout<<"\nCommon Unique Elements: ";
    for(int i=0;i<resultSize;i++){
        cout<<resultArray[i]<<" ";
    }

    delete[] resultArray;
    resultArray=NULL;
}

int *findIntersection(int *array1, int *array2, int size1, int size2, int *resultSize){
    int length;
    if(size1>size2) length=size1;
    else length=size2;
    int uCom=0;
    int common[length];
```

```

bool alreadyFound=false;
int cfilled=0;
for(int i=0;i<size1;i++){
    for(int j=0;j<size2;j++){
        if(array1[i]==array2[j]){
            for(int k=0;k<length;k++){
                if(common[k]==array1[i]){
                    alreadyFound=true;
                    break;
                }
            }
            else continue;
        }

        if(alreadyFound==true){
            alreadyFound=false;
            continue;
        }
        else{
            common[cfilled]=array1[i];
            cfilled++;
        }
    }
    else continue;
}

}

*resultSize=cfilled;
int *resultArray=new int[*resultSize];

for(int i=0;i<*resultSize;i++){
    resultArray[i]=common[i];
}

return resultArray;
}

```

Output:

```

24P-3017

A = 1 3 6 8 56
B = 1 1 6 7 8

Common Unique Elements: 1 6 8

```

Task 3:

```
#include <iostream>
using namespace std;

int main(){
    cout<<"24P-3017"<<endl;

    int r, c;
    cout<<"\nEnter number of rows: "; cin>>r;
    cout<<"Enter number of columns: "; cin>>c;

    int **arr=new int*[r];
    for(int i=0;i<r;i++){
        *(arr+i)=new int[c];
    }

    int *row=new int(0);
    int *col=new int(0);
    int counter=1;
    for(int i=0;i<r;i++){
        for(int j=0;j<c;j++){
            cout<<"Enter Elements "<<counter<<": "; counter++;
            cin>>*(*(arr+i)+j);
            if(i==j==0 || i==r-1 && j==0){
                *row+=*(*(arr+i)+j);
            }
            if(i==0 && j==c-1){
                *col+=*(*(arr+i)+j);
            }
        }
    }
    *row+=r;
    *col+=c;

    int **resultArray = new int[*row];
    for(int i=0;i<*row;i++){
        *(resultArray+i) = new int[*col]{0};
    }

    int index=0;
    for(int i=*(arr+0)+0;i<r+*(arr+0)+0;i++){
```

Output:

Output:

Task 4:

```
#include<iostream>
#include<climits>
using namespace std;
int avgStock(int *, int);
int critProducts(int *, int, int);
void topSelling(int *, int);
int findSecondBestSeller(int*, int);
void sortByPopularity(int *, int);

int main(){
    cout<<"24P-3017"<<endl;

    int invenSize;
    cout<<"\nEnter inventory size: "; cin>>invenSize;

    int *inventory=new int[invenSize];
    for(int i=0;i<invenSize;i++){
        *(inventory+i)=rand()%100;
    }

    cout<<"\nInventory: ";
    for(int i=0;i<invenSize;i++){
        cout<<*(inventory+i)<<" ";
    }

    int avgQuantity=avgStock(inventory, invenSize);
    cout<<"\nAverage Stock: "<<avgQuantity<<endl;
    cout<<"Number of Critical Products: "<<critProducts(inventory, invenSize,
avgQuantity)<<endl;
    topSelling(inventory, invenSize);
    cout<<"Product #"<<findSecondBestSeller(inventory, invenSize)<<" is the second best selling
product."<<endl;
    cout<<"\nInventory sorted by popularity: "; sortByPopularity(inventory, invenSize);

    for(int i=0;i<invenSize;i++){
        cout<<*(inventory+i)<<" ";
    }
    cout<<endl;

    delete[] inventory;
}
```

```

int avgStock(int *array, int size){
    int totalQuantity=0;
    int items=0;

    for(int i=0;i<size;i++){
        totalQuantity+=*(array+i);
        items++;
    }
    int avgQuantity=totalQuantity/items;

    return avgQuantity;
}

```

```

int critProducts(int *array, int size, int avgQuantity){
    int critProducts=0;

    for(int i=0;i<size;i++){
        if(*(array+i)<avgQuantity) critProducts++;
        else continue;
    }

    return critProducts;
}

```

```

void topSelling(int *array, int size){
    int topSellingIndex=0;
    int topSellingQuantity=INT_MAX;

    for(int i=0;i<size;i++){
        if(*(array+i)==0){
            topSellingIndex=i;
            break;
        }
        else if(*(array+i)<topSellingQuantity){
            topSellingQuantity=*(array+i);
            topSellingIndex=i;
        }
        else continue;
    }

    cout<<"Product #"<<topSellingIndex+1<<" is the top selling product."<<endl;
}

```

```

int findSecondBestSeller(int *array, int size){

```



```

int best=INT_MAX;
int secBest=INT_MAX;
int bestIndex=0;
int secBestIndex=0;

for (int i=0;i<size;i++){
    if(*(array+i)<best){
        secBest=best;
        secBestIndex=bestIndex;

        best=*(array+i);
        bestIndex=i;
    }
    else if(*(array+i)>best && *(array+i)<secBest){
        secBest=*(array+i);
        secBestIndex=i;
    }
}

return secBestIndex+1;
}

void sortByPopularity(int *array, int size){
    for(int i=0;i<size-1;i++){
        int *maxEl=(array + i);
        for(int j=i+1;j<size;j++){
            if (*(array+j)>*maxEl){
                maxEl=(array+j);
            }
        }
        if(maxEl!=(array+i)) {
            int temp=*(array+i);
            *(array+i)=*maxEl;
            *maxEl=temp;
        }
    }
}

```

Output:

24P-3017

Enter inventory size: 10

Inventory: 41 67 34 0 69 24 78 58 62 64

Average Stock: 49

Number of Critical Products: 4

Product #4 is the top selling product.

Product #6 is the second best selling product.

Inventory sorted by popularity: 78 69 67 64 62 58 41 34 24 0